

Семинарское занятие № 3

Обнаружение и исправление ошибок при передаче двоичных сообщений

Параметр	Значение
Дисциплина	Информатика
Институт	ИПТИП
Кафедра	Индустриального программирования
Вид учебного материала	Методические указания к семинарским занятиям
Преподаватель	Александров Алексей Сергеевич
Семестр	1 семестр

Материал семинарского занятия построен как последовательность действий: сначала рассматривается сама проблема искажения данных, затем простейшие способы обнаружения и исправления ошибок, после чего подробно разбирается код Хэмминга (7,4). Для кода Хэмминга используются **два равноправных способа вычислений**: через подсчёт чётности и через операцию **исключающего ИЛИ (XOR)**.

Основная структура работы соответствует теме рабочей тетради № 3 «Решение задач на обнаружение ошибок и исправление ошибок». Пояснения расширены так, чтобы материал можно было последовательно разобрать на семинарском занятии, а затем повторить самостоятельно.

Цель занятия

- понять, почему при передаче данных нужна избыточность;
- научиться обнаруживать одиночную ошибку с помощью бита чётности;
- вычислять расстояние Хэмминга между двоичными словами;
- познакомиться с исправлением ошибок методом троирования;
- понять устройство кода Хэмминга (7,4);
- научиться формировать код Хэмминга вручную;
- выполнять проверку принятого слова и определять позицию одиночной ошибки;
- увидеть связь проверки чётности с операцией XOR;
- реализовать основные алгоритмы на Python.

К концу занятия необходимо уметь объяснить, **какие биты являются информационными, какие — контрольными, зачем они добавляются и почему синдром ошибки указывает номер повреждённой позиции**.

1. Зачем вообще нужны коды обнаружения и исправления ошибок

Компьютер передаёт информацию в виде битов: 0 и 1.

Предположим, необходимо передать один бит:

1

Во время передачи из-за помех он может измениться:

1 -> 0

Получатель увидит 0. Если никакой дополнительной информации нет, он **не сможет понять**, что исходно была передана единица.

Отсюда появляется основная идея этой темы:

К полезным данным специально добавляют лишние, или избыточные, биты. Они не несут новой пользовательской информации, но позволяют проверить, не исказились ли данные при передаче.

Таким образом, приходится выбирать между двумя характеристиками:

- чем меньше дополнительных битов, тем экономнее передача;
- чем больше и лучше организована избыточность, тем надёжнее можно обнаруживать и исправлять ошибки.

В этой работе рассматриваются три уровня защиты:

бит чётности

↓

ошибку можно обнаружить, но нельзя определить её позицию

троирование

↓

ошибку можно исправить, но объём данных увеличивается втрое

код Хэмминга (7,4)

↓

одиночную ошибку можно найти и исправить при меньшей избыточности

2. Бит чётности: самый простой контроль

2.1. Идея

Пусть необходимо передать сообщение:

110101

Посчитаем количество единиц:

$1 + 1 + 0 + 1 + 0 + 1$

Количество единиц = 4

Число 4 чётное.

При **чётном контроле** к сообщению добавляется один дополнительный бит p , чтобы общее количество единиц оставалось чётным.

В данном случае ничего исправлять не требуется:

$p = 0$

Получаем кодовое слово:

1101010

2.2. Что произойдёт при ошибке

Пусть один бит изменился:

Было: 1101010

Принято: 1111010

Теперь единиц пять — нечётное количество.

Получатель делает вывод:

чётность нарушена -> ошибка обнаружена

При этом он **не знает**, какой именно бит неверен.

2.3. Ограничение бита чётности

Если одновременно изменятся два бита, чётность может снова оказаться правильной.

Поэтому один бит чётности:

- обнаруживает любую одиночную ошибку;
- обнаруживает ошибки нечётной кратности;
- не показывает позицию ошибки;
- не гарантирует обнаружение ошибки чётной кратности.

2.4. Python

```
message = "110101"

ones = message.count("1")
p = ones % 2
codeword = message + str(p)

print("Сообщение:", message)
print("Количество единиц:", ones)
print("Бит чётности:", p)
print("Кодовое слово:", codeword)
```

Для 110101 результат:

Сообщение: 110101
Количество единиц: 4
Бит чётности: 0
Кодовое слово: 1101010

3. Расстояние Хэмминга

Расстояние Хэмминга — это количество позиций, в которых два двоичных слова одинаковой длины различаются.

Возьмём:

1010110
1000011

Сравним позиции:

Позиция:	1	2	3	4	5	6	7
Слово 1:	1	0	1	0	1	1	0
Слово 2:	1	0	0	0	0	1	1
			^		^		^

Различаются позиции 3, 5 и 7.

Следовательно:

$d = 3$

3.1. Практический смысл

Предположим, допустимы только два кодовых слова:

000
111

Между ними расстояние Хэмминга равно 3.

Если было отправлено 000, а принято:

010

то:

расстояние до 000 = 1
расстояние до 111 = 2

Принятое слово ближе к 000, поэтому можно предположить, что исходно было передано именно оно.

Общая идея:

Чем дальше друг от друга расположены допустимые кодовые слова, тем больше искажений требуется, чтобы одно допустимое слово стало похоже на другое.

3.2. Python

```
a = "1010110"  
b = "1000011"  
  
d = sum(x != y for x, y in zip(a, b))  
  
print("Слово 1:", a)  
print("Слово 2:", b)  
print("Расстояние Хэмминга:", d)
```

Результат:

Расстояние Хэмминга: 3

4. Троирование: простое исправление ошибки

Бит чётности умеет обнаруживать ошибку, но не умеет её исправлять.

Самый очевидный способ сделать передачу устойчивее — отправлять каждый бит несколько раз.

При троировании:

0 -> 000
1 -> 111

Для сообщения:

1010

получаем:

111 000 111 000

Пусть в первой тройке произошла одиночная ошибка:

111 -> 101

Получатель видит:

101

В этой тройке две единицы и один ноль.

По правилу большинства выбирается значение, которое встречается чаще:

101 -> 1

Исходный бит восстановлен.

4.1. Недостаток троирования

Исходное сообщение:

1010

занимает 4 бита.

После троирования:

111000111000

получается уже 12 бит.

Объём данных увеличился втрое.

Возникает естественный вопрос:

Можно ли научиться исправлять одиночную ошибку, добавляя меньше лишних битов?

Для этого используется код Хэмминга.

4.2. Python

```
message = "1010"
```

```
encoded = "".join("111" if bit == "1" else "000" for bit in message)
```

```
broken = list(encoded)
broken[1] = "0" if broken[1] == "1" else "1"
broken = "".join(broken)
```

```
def majority(triple: str) -> str:
    return "1" if triple.count("1") >= 2 else "0"
```

```
decoded = "".join(
    majority(broken[i:i + 3])
    for i in range(0, len(broken), 3)
)
```

```
print("Исходное сообщение:", message)
print("Троированный код:", encoded)
print("Принято с ошибкой:", broken)
print("Декодировано:", decoded)
```

5. Код Хэмминга (7,4): что означает запись (7,4)

Код Хэмминга (7,4) работает с блоками из четырёх исходных битов.

Запись (7,4) означает:

4 информационных бита
+
3 контрольных бита
=
7 бит после кодирования

Важно:

Из четырёх исходных битов не «вычисляются» семь новых битов. Четыре исходных бита переписываются как есть, а к ним добавляются ещё три специально рассчитанных контрольных бита.

5.1. Почему именно 4 информационных бита и 7 бит всего

Числа 7 и 4 не выбраны случайно. Они получаются из того, сколько разных ситуаций должен уметь различить код при проверке.

Пусть требуется передавать $k = 4$ информационных бита. Добавим r контрольных битов. Тогда всего после кодирования будет:

$$n = k + r$$

После передачи возможны следующие ситуации:

ошибки нет;
ошибка в позиции 1;
ошибка в позиции 2;
...
ошибка в позиции n .

То есть проверка должна различать $n + 1$ состояний: одно состояние «ошибки нет» и по одному состоянию для каждой возможной ошибочной позиции.

Если контрольных битов r , результат проверки — синдром длиной r бит. Из r бит можно получить 2^r различных комбинаций. Поэтому необходимо условие:

$$2^r \geq n + 1$$

Так как $n = k + r$, получаем:

$$2^r \geq k + r + 1$$

Теперь подставим $k = 4$.

Если взять только два контрольных бита:

$$\begin{aligned} r &= 2 \\ 2^2 &= 4 \\ 4 + 2 + 1 &= 7 \\ 4 &\geq 7 \quad - \text{ неверно} \end{aligned}$$

Двух контрольных битов недостаточно: четыре возможных синдрома не смогут закодировать все варианты.

Берём три контрольных бита:

$$\begin{aligned} r &= 3 \\ 2^3 &= 8 \\ 4 + 3 + 1 &= 8 \\ 8 &\geq 8 \quad - \text{ верно} \end{aligned}$$

Получается ровно восемь возможных синдромов:

000 — ошибки нет
001 — ошибка в позиции 1
010 — ошибка в позиции 2

011 – ошибка в позиции 3
 100 – ошибка в позиции 4
 101 – ошибка в позиции 5
 110 – ошибка в позиции 6
 111 – ошибка в позиции 7

Поэтому для четырёх информационных битов достаточно трёх контрольных:

$$4 + 3 = 7$$

Отсюда и обозначение:

Хэмминг (7,4)
 | |
 | +-- 4 информационных бита
 +---- 7 бит всего после кодирования

5.2. Могут ли быть другие коды Хэмминга

Да. (7,4) — только самый удобный небольшой вариант для изучения вручную.

Для полного кода Хэмминга с r контрольными битами:

$$n = 2^r - 1$$

$$k = 2^r - r - 1$$

Примеры:

Контрольных битов r	Всего битов n	Информационных битов k	Обозначение
2	3	1	(3,1)
3	7	4	(7,4)
4	15	11	(15,11)
5	31	26	(31,26)

Принцип остаётся тем же: контрольные биты располагаются в позициях 1, 2, 4, 8, 16, ..., то есть в степенях двойки. Каждый такой бит отвечает за один разряд двоичного номера позиции.

Существуют также укороченные варианты: не обязательно использовать все доступные информационные позиции. Кроме того, к обычному коду Хэмминга можно добавить ещё один общий бит чётности. Например, из (7,4) получается расширенный вариант (8,4), который позволяет надёжно отличать одиночную ошибку от двойной: одиночную исправить, а двойную обнаружить. Такой режим часто обозначают **SEDED** — *Single Error Correction, Double Error Detection*.

Возьмём исходное сообщение:

1011

Дадим исходным битам имена:

$d_1 = 1$
 $d_2 = 0$
 $d_3 = 1$
 $d_4 = 1$

Буква d означает **data**, то есть данные.

Это просто имена исходных битов. Никаких вычислений здесь нет.

6. Информационные и контрольные биты

6.1. Информационные биты

Информационные биты — это полезные данные, которые требуется передать.

В нашем примере:

1011

то есть:

$d1 = 1$

$d2 = 0$

$d3 = 1$

$d4 = 1$

6.2. Контрольные биты

Контрольные биты — дополнительные биты, которые добавляются специально для проверки правильности передачи.

В коде Хэмминга (7,4) их три:

$p1$

$p2$

$p4$

Они занимают позиции:

1, 2, 4

Это степени двойки:

$1 = 2^0$

$2 = 2^1$

$4 = 2^2$

Остальные позиции используются для данных:

3, 5, 6, 7

Полная схема:

Позиция:	1	2	3	4	5	6	7
	$p1$	$p2$	$d1$	$p4$	$d2$	$d3$	$d4$

Теперь просто переписываем исходное 1011 на места $d1$, $d2$, $d3$, $d4$:

Позиция:	1	2	3	4	5	6	7
	?	?	1	?	0	1	1

Вопросительные знаки — это три контрольных бита, которые ещё необходимо вычислить.

7. Первый способ: вычисляем контрольные биты через чётность

Для чётного контроля значение каждого p выбирается так, чтобы в его группе получилось **чётное количество единиц**.

Группы в коде Хэмминга (7,4):

p1 проверяет позиции 1, 3, 5, 7
p2 проверяет позиции 2, 3, 6, 7
p4 проверяет позиции 4, 5, 6, 7

Пока просто используем эти группы как правило. Почему они устроены именно так, будет разобрано далее.

7.1. Вычисляем p1

p1 проверяет позиции:

1, 3, 5, 7

В них находятся:

p1, d1, d2, d4

Подставим данные:

p1, 1, 0, 1

Без p1 уже есть две единицы.

Две — чётное число, поэтому:

p1 = 0

Получаем:

0, 1, 0, 1

Количество единиц осталось чётным.

7.2. Вычисляем p2

p2 проверяет:

2, 3, 6, 7

То есть:

p2, d1, d3, d4

Подставляем:

p2, 1, 1, 1

Без p2 три единицы — нечётное число.

Чтобы количество стало чётным:

p2 = 1

Получается четыре единицы.

7.3. Вычисляем p4

p4 проверяет:

4, 5, 6, 7

То есть:

p4, d2, d3, d4

Подставляем:

p4, 0, 1, 1

Уже есть две единицы — чётное количество.

Следовательно:

$p4 = 0$

7.4. Собираем кодовое слово

Мы получили:

$p1 = 0$

$p2 = 1$

$p4 = 0$

Подставляем в семь позиций:

Позиция: 1 2 3 4 5 6 7
 0 1 1 0 0 1 1

Итог:

1011 -> 0110011

Четыре исходных бита сохранились внутри слова, а три дополнительных бита обеспечивают контроль.

8. Второй способ: те же вычисления через XOR

Операция XOR — **исключающее ИЛИ**.

Для двух битов:

A	B	A XOR B
0	0	0
0	1	1
1	0	1
1	1	0

Для нескольких битов XOR удобно интерпретировать как проверку количества единиц:

чётное количество единиц -> XOR = 0

нечётное количество единиц -> XOR = 1

Примеры:

$1 \text{ XOR } 0 \text{ XOR } 1 = 0$ # две единицы

$1 \text{ XOR } 1 \text{ XOR } 1 = 1$ # три единицы

Поэтому подсчёт чётности и XOR — не два разных алгоритма. Это **два способа выполнить одну и ту же проверку**.

8.1. p1 через XOR

Для p1 используются d1, d2, d4:

$p1 = d1 \text{ XOR } d2 \text{ XOR } d4$

Подставляем:

```
p1 = 1 XOR 0 XOR 1
p1 = 0
```

8.2. p2 через XOR

```
p2 = d1 XOR d3 XOR d4
p2 = 1 XOR 1 XOR 1
p2 = 1
```

8.3. p4 через XOR

```
p4 = d2 XOR d3 XOR d4
p4 = 0 XOR 1 XOR 1
p4 = 0
```

Получили те же значения:

```
p1 = 0
p2 = 1
p4 = 0
```

и то же кодовое слово:

0110011

В Python XOR для целых чисел записывается оператором ^:

```
p1 = d1 ^ d2 ^ d4
p2 = d1 ^ d3 ^ d4
p4 = d2 ^ d3 ^ d4
```

9. Почему p1, p2 и p4 проверяют именно такие позиции

Это главная идея кода Хэмминга.

Пронумеруем позиции от 1 до 7 и запишем их номера в двоичном виде:

Позиция	Двоичный номер
1	001
2	010
3	011
4	100
5	101
6	110
7	111

Теперь посмотрим на двоичные разряды.

9.1. Группа p1

p1 отвечает за младший разряд — значение 1.

Берём все позиции, у которых младший разряд двоичного номера равен единице:

1 = 001
3 = 011
5 = 101
7 = 111

Поэтому:

p1 -> 1, 3, 5, 7

9.2. Группа p2

p2 отвечает за разряд со значением 2:

2 = 010
3 = 011
6 = 110
7 = 111

Поэтому:

p2 -> 2, 3, 6, 7

9.3. Группа p4

p4 отвечает за разряд со значением 4:

4 = 100
5 = 101
6 = 110
7 = 111

Поэтому:

p4 -> 4, 5, 6, 7

Именно благодаря этому набор сработавших проверок позже образует **двоичный номер повреждённого бита**.

10. Как выполняется проверка принятого слова

Отправляем корректное кодовое слово:

0110011

Предположим, во время передачи изменился бит в позиции 5.

Было:

Позиция:	1	2	3	4	5	6	7
	0	1	1	0	0	1	1

Стало:

Позиция:	1	2	3	4	5	6	7
	0	1	1	0	1	1	1
					^		

Получатель получает:

0110111

Он не знает, где находится ошибка. Поэтому снова выполняет три проверки.

10.1. Проверка первым способом: считаем единицы

Проверка s1

Группа p1:

позиции 1, 3, 5, 7

Биты:

0, 1, 1, 1

Единиц три — нечётное количество.

Следовательно:

$s1 = 1$

Первая проверка сообщает: в её группе есть нарушение.

Проверка s2

Группа p2:

позиции 2, 3, 6, 7

Биты:

1, 1, 1, 1

Единиц четыре — чётное количество.

$s2 = 0$

Проверка s4

Группа p4:

позиции 4, 5, 6, 7

Биты:

0, 1, 1, 1

Единиц три — нечётное количество.

$s4 = 1$

Итог:

$s1 = 1$

$s2 = 0$

$s4 = 1$

10.2. Проверка вторым способом: XOR

Вместо подсчёта единиц можно выполнить XOR всех битов каждой группы.

Для s1:

$s1 = 0 \text{ XOR } 1 \text{ XOR } 1 \text{ XOR } 1$

$s1 = 1$

Для s2:

$s_2 = 1 \oplus 1 \oplus 1 \oplus 1$
 $s_2 = 0$

Для s_4 :

$s_4 = 0 \oplus 1 \oplus 1 \oplus 1$
 $s_4 = 1$

Результат полностью совпадает:

$s_1 = 1$
 $s_2 = 0$
 $s_4 = 1$

11. Синдром: откуда появляется номер ошибки

Результаты трёх проверок называются **синдромом ошибки**.

Каждая проверка имеет свой вес:

$s_1 \rightarrow 1$
 $s_2 \rightarrow 2$
 $s_4 \rightarrow 4$

Получено:

$s_1 = 1$
 $s_2 = 0$
 $s_4 = 1$

Можно вычислить номер позиции:

$1 * 1 + 0 * 2 + 1 * 4 = 5$

Или записать проверки как двоичное число в порядке $s_4 \ s_2 \ s_1$:

$s_4 \ s_2 \ s_1$
1 0 1

Получаем:

$101_2 = 5_{10}$

То есть:

ошибка в позиции 5

Это не совпадение.

Позиция 5 в двоичном виде:

$5 = 101_2$

Она входит:

в группу $p_1 \rightarrow$ да
в группу $p_2 \rightarrow$ нет
в группу $p_4 \rightarrow$ да

Именно поэтому сработавшие проверки дают:

1 0 1

То есть двоичный номер позиции.

Ещё два коротких примера

Если испортилась позиция 3:

$$3 = 011_2$$

сработают p_1 и p_2 , но не p_4 :

$$s_4 \ s_2 \ s_1 = 0 \ 1 \ 1 = 3$$

Если испортилась позиция 6:

$$6 = 110_2$$

сработают p_2 и p_4 :

$$s_4 \ s_2 \ s_1 = 1 \ 1 \ 0 = 6$$

Если синдром равен нулю:

$$000_2 = 0$$

одиночная ошибка не обнаружена.

12. Исправление ошибки

После того как синдром указал позицию 5, остаётся изменить этот бит на противоположный.

Принято:

$$0110111$$

Ошибка в позиции 5:

$$1 \rightarrow 0$$

После исправления:

$$0110011$$

Это исходное кодовое слово.

Чтобы получить исходные пользовательские данные, берём только информационные позиции:

3, 5, 6, 7

Из исправленного слова:

$$\begin{array}{ccccccc} 0 & 1 & 1 & 0 & 0 & 1 & 1 \\ & \wedge & & \wedge & \wedge & \wedge & \end{array}$$

получаем:

$$1 \ 0 \ 1 \ 1$$

то есть исходное сообщение:

$$1011$$

13. Полный алгоритм Хэмминга (7,4)

Кодирование

Для четырёх исходных битов:

d1 d2 d3 d4

выполните следующие действия.

1. Создайте семь позиций.
2. Зарезервируйте позиции 1, 2, 4 под контрольные биты.
3. Запишите данные в позиции 3, 5, 6, 7.
4. Вычислите p1 по группе 1, 3, 5, 7.
5. Вычислите p2 по группе 2, 3, 6, 7.
6. Вычислите p4 по группе 4, 5, 6, 7.
7. Передайте полученное 7-битное слово.

Схема:

Исходные данные

```
1011
  |
  v
d1=1 d2=0 d3=1 d4=1
  |
  v
p1 p2 d1 p4 d2 d3 d4
?  ?  1  ?  0  1  1
  |
  v
вычисляем p1, p2, p4
  |
  v
0  1  1  0  0  1  1
  |
  v
0110011
```

Проверка и исправление

1. Получите семь битов.
2. Выполните проверку группы 1, 3, 5, 7 и получите s1.
3. Выполните проверку группы 2, 3, 6, 7 и получите s2.
4. Выполните проверку группы 4, 5, 6, 7 и получите s4.
5. Вычислите синдром:

$\text{syndrome} = s1 * 1 + s2 * 2 + s4 * 4$

6. Если syndrome = 0, одиночная ошибка не обнаружена.
7. Если syndrome != 0, это номер ошибочной позиции.
8. Инвертируйте найденный бит.
9. Возьмите информационные биты из позиций 3, 5, 6, 7.

14. Реализация кода Хэмминга на Python

14.1. Кодирование

```
def hamming74_encode(data_bits: str) -> str:
    if len(data_bits) != 4 or any(bit not in "01" for bit in data_bits):
        raise ValueError("Ожидается строка из четырёх битов")

    d1, d2, d3, d4 = (int(bit) for bit in data_bits)
```

```

p1 = d1 ^ d2 ^ d4
p2 = d1 ^ d3 ^ d4
p4 = d2 ^ d3 ^ d4

code = [p1, p2, d1, p4, d2, d3, d4]
return "".join(str(bit) for bit in code)

```

Проверка:

```
print(hamming74_encode("1011"))
```

Результат:

0110011

14.2. Вычисление синдрома

```

def hamming74_syndrome(code_bits: str) -> int:
    if len(code_bits) != 7 or any(bit not in "01" for bit in code_bits):
        raise ValueError("Ожидается строка из семи битов")

    b = [int(bit) for bit in code_bits]

    s1 = b[0] ^ b[2] ^ b[4] ^ b[6]
    s2 = b[1] ^ b[2] ^ b[5] ^ b[6]
    s4 = b[3] ^ b[4] ^ b[5] ^ b[6]

    return s1 * 1 + s2 * 2 + s4 * 4

```

Для повреждённого слова:

```
print(hamming74_syndrome("0110111"))
```

получаем:

5

14.3. Исправление

```

def hamming74_correct(code_bits: str) -> str:
    b = [int(bit) for bit in code_bits]
    position = hamming74_syndrome(code_bits)

    if position != 0:
        b[position - 1] ^= 1

    return "".join(str(bit) for bit in b)

```

Здесь используется строка:

```
b[position - 1] ^= 1
```

Она меняет найденный бит на противоположный:

0 XOR 1 = 1

1 XOR 1 = 0

То есть ^= 1 удобно использовать как операцию инверсии бита.

14.4. Извлечение исходных данных

```
def hamming74_decode(code_bits: str) -> str:
    fixed = hamming74_correct(code_bits)
    return fixed[2] + fixed[4] + fixed[5] + fixed[6]
```

Информационные позиции 3, 5, 6, 7 в Python имеют индексы:

2, 4, 5, 6

потому что нумерация списка начинается с нуля.

14.5. Полный пример передачи

```
def hamming74_encode(data_bits: str) -> str:
    d1, d2, d3, d4 = (int(bit) for bit in data_bits)

    p1 = d1 ^ d2 ^ d4
    p2 = d1 ^ d3 ^ d4
    p4 = d2 ^ d3 ^ d4

    return f"{p1}{p2}{d1}{p4}{d2}{d3}{d4}"
```

```
def hamming74_syndrome(code_bits: str) -> int:
    b = [int(bit) for bit in code_bits]

    s1 = b[0] ^ b[2] ^ b[4] ^ b[6]
    s2 = b[1] ^ b[2] ^ b[5] ^ b[6]
    s4 = b[3] ^ b[4] ^ b[5] ^ b[6]

    return s1 + 2 * s2 + 4 * s4
```

```
def hamming74_correct(code_bits: str) -> str:
    b = [int(bit) for bit in code_bits]
    position = hamming74_syndrome(code_bits)

    if position != 0:
        b[position - 1] ^= 1

    return "".join(str(bit) for bit in b)
```

```
def hamming74_decode(code_bits: str) -> str:
    fixed = hamming74_correct(code_bits)
    return fixed[2] + fixed[4] + fixed[5] + fixed[6]
```

```
data = "1011"
code = hamming74_encode(data)
```

```
broken = list(code)
broken[4] = "0" if broken[4] == "1" else "1"
```

```

broken = "".join(broken)

syndrome = hamming74_syndrome(broken)
fixed = hamming74_correct(broken)
decoded = hamming74_decode(broken)

print("Исходные данные:      ", data)
print("Код Хэмминга:        ", code)
print("Принято с ошибкой:     ", broken)
print("Синдром:               ", syndrome)
print("Исправленное слово:     ", fixed)
print("Восстановленные данные:", decoded)

```

Ожидаемый результат:

```

Исходные данные:      1011
Код Хэмминга:         0110011
Принято с ошибкой:    0110111
Синдром:              5
Исправленное слово:   0110011
Восстановленные данные: 1011

```

15. Практическое задание

Работа выполняется последовательно. Для основных вычислений сначала используйте ручной способ, затем подтвердите результат программно.

Задание 1. Бит чётности

Для сообщения:

101011

1. Посчитайте количество единиц.
2. Определите бит чётности при чётном контроле.
3. Сформируйте кодовое слово.
4. Напишите программу, выполняющую тот же расчёт.
5. Искусственно измените один бит и проверьте, обнаруживается ли ошибка.
6. Измените два бита и объясните результат проверки.

Задание 2. Расстояние Хэмминга

Выберите два двоичных слова одинаковой длины не менее 7 бит.

1. Выпишите их друг под другом.
2. Отметьте отличающиеся позиции.
3. Вычислите расстояние вручную.
4. Подтвердите результат программно.
5. Объясните, что означает полученное значение.

Задание 3. Троирование

Для произвольного четырёхбитного сообщения:

1. Выполните троирование каждого бита.
2. Определите, во сколько раз вырос объём данных.

3. Измените один бит внутри одной тройки.
4. Восстановите сообщение правилом большинства.
5. Реализуйте кодирование и декодирование на Python.

Задание 4. Код Хэмминга вручную

Используйте сообщение:

1101

1. Обозначьте исходные биты как d_1, d_2, d_3, d_4 .
2. Разместите их в позициях 3, 5, 6, 7.
3. Вычислите p_1, p_2, p_4 через подсчёт чётности.
4. Повторите вычисление через XOR.
5. Убедитесь, что оба способа дали одинаковый результат.
6. Запишите итоговое семибитное слово.

Задание 5. Поиск ошибки

Возьмите кодовое слово из задания 4.

1. Измените бит в позиции 6.
2. Выполните проверки s_1, s_2, s_4 вручную через чётность.
3. Выполните те же проверки через XOR.
4. Запишите синдром как $s_4 s_2 s_1$.
5. Переведите двоичный синдром в десятичное число.
6. Сравните его с номером позиции, которую вы изменили.
7. Исправьте бит.
8. Восстановите четыре исходных информационных бита.

Задание 6. Программная реализация

Реализуйте функции:

```
hamming74_encode(data_bits)
hamming74_syndrome(code_bits)
hamming74_correct(code_bits)
hamming74_decode(code_bits)
```

Программа должна выводить:

исходное сообщение;
кодовое слово;
принятое слово после моделирования ошибки;
синдром;
номер ошибочной позиции;
слово после исправления;
восстановленные информационные биты.

Проверьте программу, последовательно создавая ошибку в каждой позиции от 1 до 7.
Для каждой позиции убедитесь, что синдром совпадает с номером изменённого бита.

16. Повышенный уровень

16.1. Экспериментальное сравнение методов

Программно сравните:

- бит чётности;
- троирование;
- код Хэмминга (7,4).

Сгенерируйте серию случайных четырёхбитных сообщений, в каждом случае внесите единичную ошибку в случайную позицию и подсчитайте:

- сколько ошибок удалось обнаружить;
- сколько ошибок удалось исправить;
- сколько дополнительных битов потребовал каждый метод.

Сформулируйте вывод о компромиссе между избыточностью и возможностью исправления.

16.2. Модуль передачи сообщений

Создайте программный модуль, содержащий отдельные функции для:

- кодирования;
- моделирования ошибки;
- проверки;
- исправления;
- декодирования;
- печати промежуточных этапов.

Пользователь должен иметь возможность ввести исходное сообщение и увидеть весь путь от исходных данных до восстановленного результата.

17. Достаточно ли этих методов для реальных систем

Методы из этой работы реальны, но они выбраны прежде всего потому, что на них удобно увидеть основные идеи кодирования.

Бит чётности

Бит чётности действительно применяется как простой механизм контроля, однако его возможности ограничены. Он показывает, что в группе изменилось нечётное число битов, но не сообщает позицию ошибки и может пропустить ошибки чётной кратности.

Троирование

Троирование хорошо показывает идею исправления по большинству, но буквально передавать каждый бит три раза слишком дорого: объём данных увеличивается втрое. В реальных системах стараются получить такую же или более высокую надёжность при значительно меньшей избыточности.

Код Хэмминга

Обычный код Хэмминга хорошо подходит для исправления одной ошибки в блоке. Расширенные варианты с дополнительным общим битом чётности позволяют реализовать SECDED: исправление одной ошибки и обнаружение двух. Подобная идея используется в системах коррекции ошибок памяти и других цифровых устройствах, хотя конкретные современные реализации могут использовать более сложные коды.

Что используется, когда ошибок может быть больше

Для реальных каналов связи и накопителей часто требуются более сильные методы. В зависимости от задачи применяются, например:

- **CRC** — прежде всего для надёжного обнаружения ошибок;
- **ВСН-коды** — для исправления нескольких битовых ошибок;
- **коды Рида-Соломона** — хорошо работают с пакетами и группами повреждений, используются в системах хранения и кодах визуальной маркировки;
- **свёрточные и Turbo-коды** — применялись и применяются в системах связи;
- **LDPC-коды** — используются в современных высокоскоростных системах связи;
- **полярные коды** — применяются в современных телекоммуникационных стандартах.

Для курса информатики подробно изучать устройство всех этих кодов не требуется. Здесь важнее понять общий принцип, который остаётся тем же:

полезные данные

+

контролируемая избыточность

↓

возможность обнаружить или исправить искажение

Код Хэмминга удобен именно потому, что на семи битах этот принцип можно полностью проследить вручную: от добавления контрольных битов до получения синдрома и исправления повреждённой позиции.

18. Что должно быть понятно после выполнения работы

После выполнения работы необходимо уметь своими словами объяснить следующие положения.

1. Почему из 4 бит получается 7?

Потому что четыре исходных информационных бита сохраняются и к ним добавляются три контрольных.

$$4 + 3 = 7$$

2. Откуда берутся d1, d2, d3, d4?

Это просто имена четырёх исходных битов. Они не вычисляются.

$$1011 \rightarrow d1=1, d2=0, d3=1, d4=1$$

3. Что такое p1, p2, p4?

Это дополнительные контрольные биты, значения которых вычисляются по группам позиций.

4. Почему контрольные позиции — 1, 2 и 4?

Потому что это степени двойки. Каждый контрольный бит соответствует одному разряду двоичного номера позиции.

5. Почему p1 проверяет 1, 3, 5, 7?

Потому что у двоичных номеров этих позиций младший разряд равен единице.

6. Что делает XOR?

XOR удобно проверяет чётность:

чётное число единиц -> 0

нечётное число единиц -> 1

7. Что такое синдром?

Это результаты контрольных проверок s_1 , s_2 , s_4 , из которых получается двоичный номер повреждённой позиции.

8. Почему синдром 101 означает ошибку в позиции 5?

Потому что:

$$101_2 = 5_{10}$$

и набор сработавших проверок соответствует двоичному номеру пятой позиции.

19. Вопросы для самопроверки

1. Зачем в кодирование вводится избыточность?
 2. Что умеет бит чётности и чего он не умеет?
 3. Что показывает расстояние Хэмминга?
 4. Как работает правило большинства при троировании?
 5. Что означают числа 7 и 4 в обозначении Хэмминга (7,4)?
 6. Какие позиции занимают информационные биты?
 7. Какие позиции занимают контрольные биты?
 8. В чём связь XOR и проверки чётности?
 9. Почему группы p_1 , p_2 , p_4 устроены именно так?
 10. Что такое синдром ошибки?
 11. Как по синдрому определить позицию повреждённого бита?
 12. Что означает синдром 0?
 13. Почему для четырёх информационных битов достаточно именно трёх контрольных?
 14. Как из числа контрольных битов получить размеры полного кода Хэмминга?
 15. Почему код Хэмминга (7,4) не является универсальным решением для всех реальных каналов связи?
-

Итог

Общая логика работы выглядит следующим образом:

Исходные данные

|
v

Добавляем избыточность

|
v

Получаем кодовое слово

|
v

Передаём данные

|
v

В канале может возникнуть ошибка

|

v

Выполняем контрольные проверки

|

v

Получаем синдром

|

v

Находим позицию одиночной ошибки

|

v

Исправляем бит

|

v

Восстанавливаем исходные данные

Главная идея кода Хэмминга состоит в том, что контрольные группы организованы не случайно: каждая из них отвечает за один разряд двоичного номера позиции. Поэтому комбинация нарушенных проверок сама образует номер повреждённого бита.